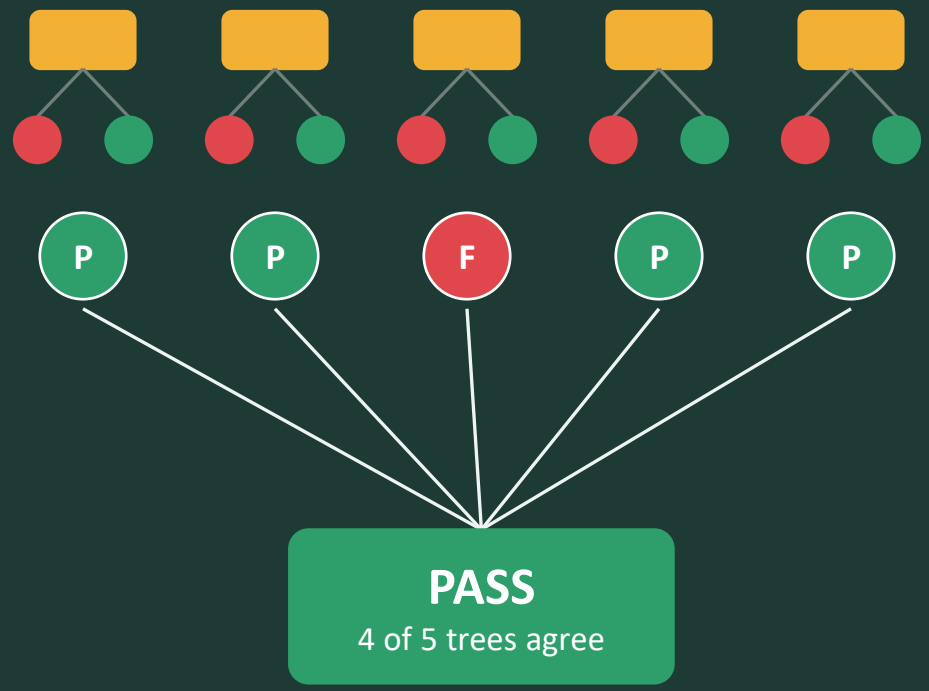
Random Forests

Why a crowd of trees beats one clever tree



Practical Machine Learning

Guess the weight of the ox

England, 1906. At a country fair, hundreds of visitors paid to guess the weight of an ox.

787

guesses collected

1,207 lb

the middle guess of the crowd

1,198 lb

the ox’s actual weight

Individual guesses were all over the place. The crowd’s answer was off by less than 1%.

Try it now

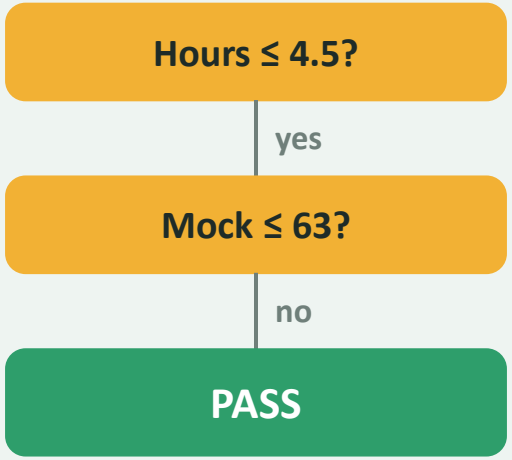
Everyone writes down a guess for the number of sweets in a jar. Compare the closest individual with the class average.

Remember Riya? One tree is fragile

Same algorithm, same 12 students, slightly different samples, and Riya gets different answers.

Last lesson's tree

All 12 students



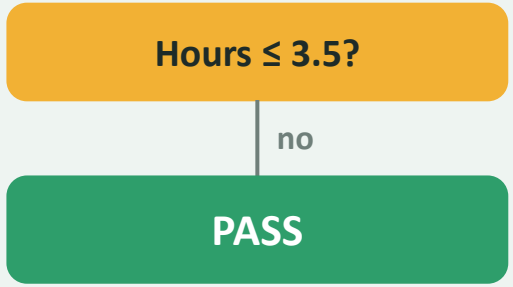
Tree from sample B

A random resample of the 12



Tree from sample C

Another random resample



This is high variance: small changes in the data give a different tree, and a different answer.

Our route today

Five stops, each answering the question the previous one leaves open.

1

Ensembles

Why do crowds beat individuals?

2

Bagging

How do we get different trees from one dataset?

3

Forest

What makes a forest random?

4

Regression

The same idea for predicting numbers

5

In practice

scikit-learn, settings and limits

The forest votes on Riya

Stop 1 of 5

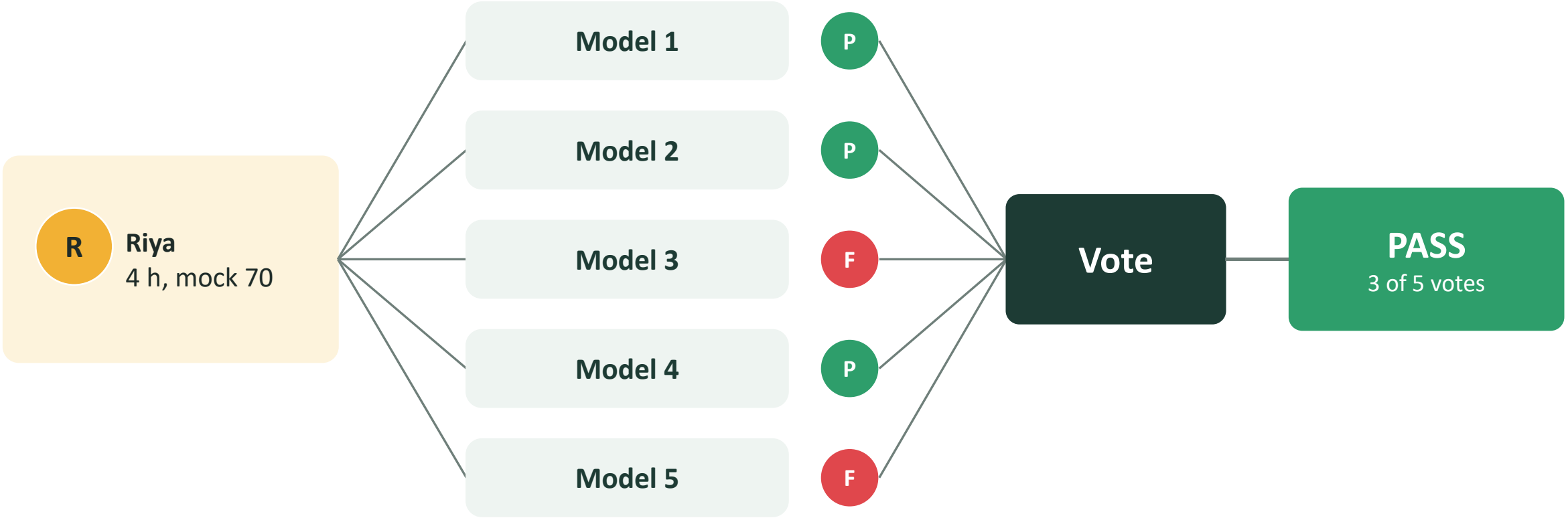
Why do crowds beat individuals?

1

An ensemble combines many models into one answer. Let’s see why that works, and when it doesn’t.

An ensemble: many models, one answer

Each model makes its own prediction. A simple rule combines them.



Classification: majority vote. Regression: average. You already trust ensembles: a panel of judges, a second medical opinion.

Why voting works: a worked example

Three independent models, each right 70% of the time. How often is the majority right?

Outcome	Working	Probability
All 3 right	0.7 × 0.7 × 0.7	0.343
Exactly 2 right	3 ways × (0.7 × 0.7 × 0.3)	0.441
Majority right	**0.343 + 0.441**	**0.784**

The 3 ways for exactly two to be right: models 1 and 2, 1 and 3, or 2 and 3.

One model

70%

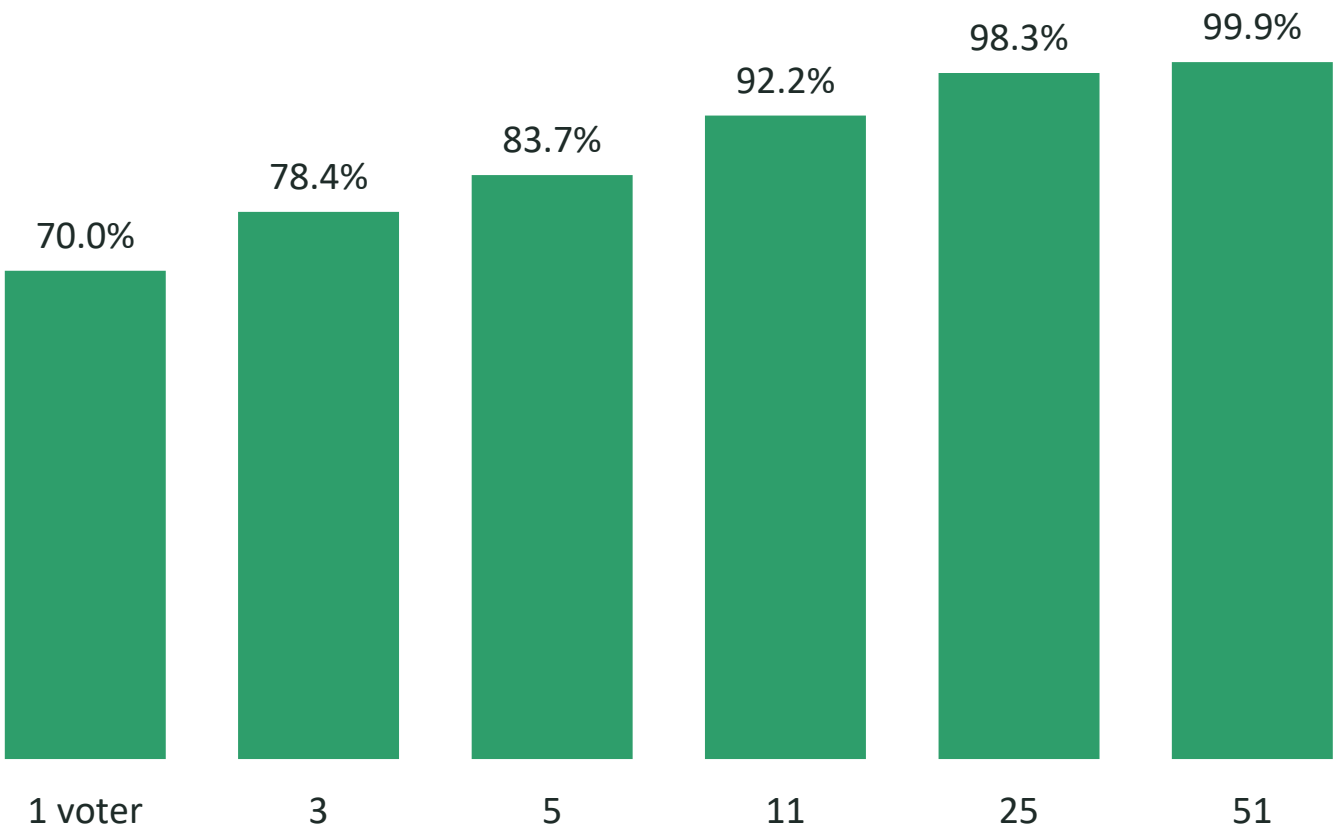
Majority of three

78.4%

Same quality models, better team.

More voters, better majority

Majority-vote accuracy when every voter is independently right 70% of the time.



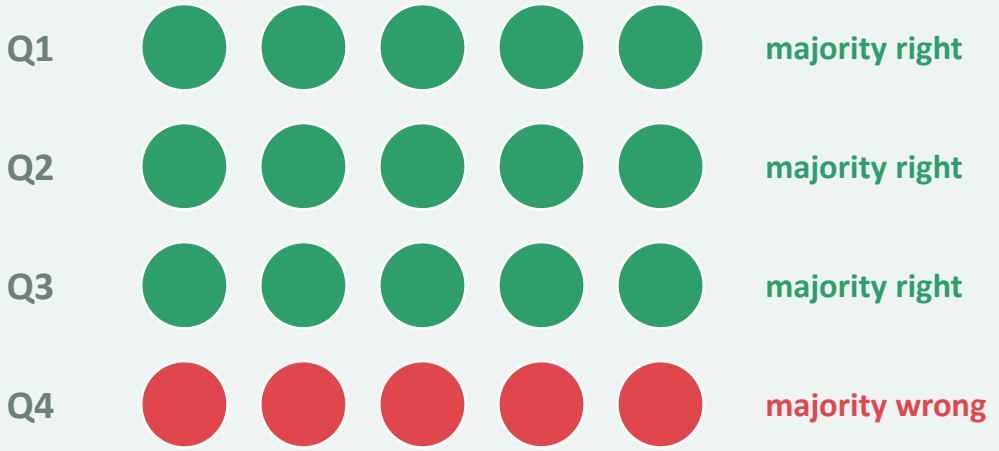
Read the small print

These numbers assume every voter makes independent mistakes. Real models trained on similar data tend to make the same mistakes, so real gains are smaller. Still, the more different the voters, the bigger the gain.

The catch: voters must make different mistakes

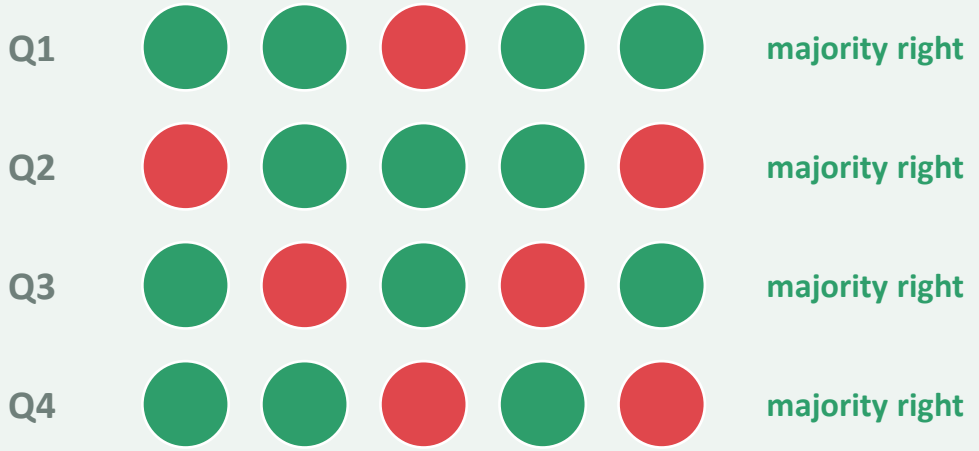
Five voters answer four questions. Green = right, red = wrong.

5 copies of one model



Majority right on 3 of 4. No better than one model.

5 different models



Majority right on 4 of 4. Mistakes cancel out.

Three families of ensembles

Today is about bagging. Random Forest is its most famous member.

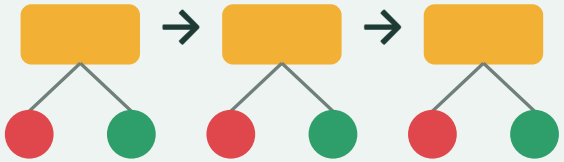
Bagging



Many models trained side by side, each on a different random sample. Combine by vote or average.

Random Forest lives here

Boosting



Models trained one after another. Each new model concentrates on the previous ones' mistakes.

Next lesson

Voting and stacking



Different kinds of model (tree, KNN, logistic regression) combined by a vote, or by another model.

Mixes model types

Stop 2 of 5

How do we get different trees from one dataset?

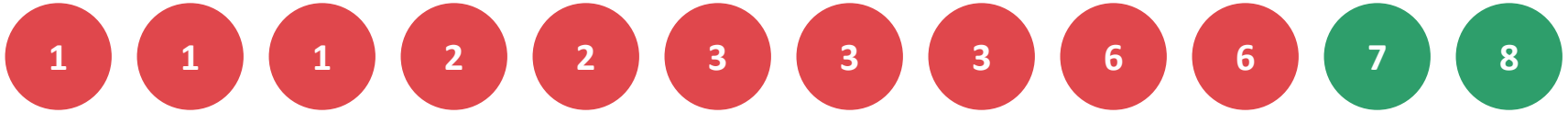
2

We only have 12 students. The trick: give every tree its own random version of them.

Bootstrap sampling: draw with replacement

Put the 12 students in a hat. Draw one, note the name, put them back. Repeat 12 times.

Sample B: 12 draws (numbers are student IDs)



Never drawn



Same size as the original, but different. Students 1 and 3 count three times, students 2 and 6 twice, and six students vanished. Each tree in the forest gets its own sample like this.

About a third are left out every time

What's the chance that one particular student is never drawn?

P(never drawn in 12 draws) = $(11/12)^{12}$ ≈ 0.35

≈ 63% of students are in each sample

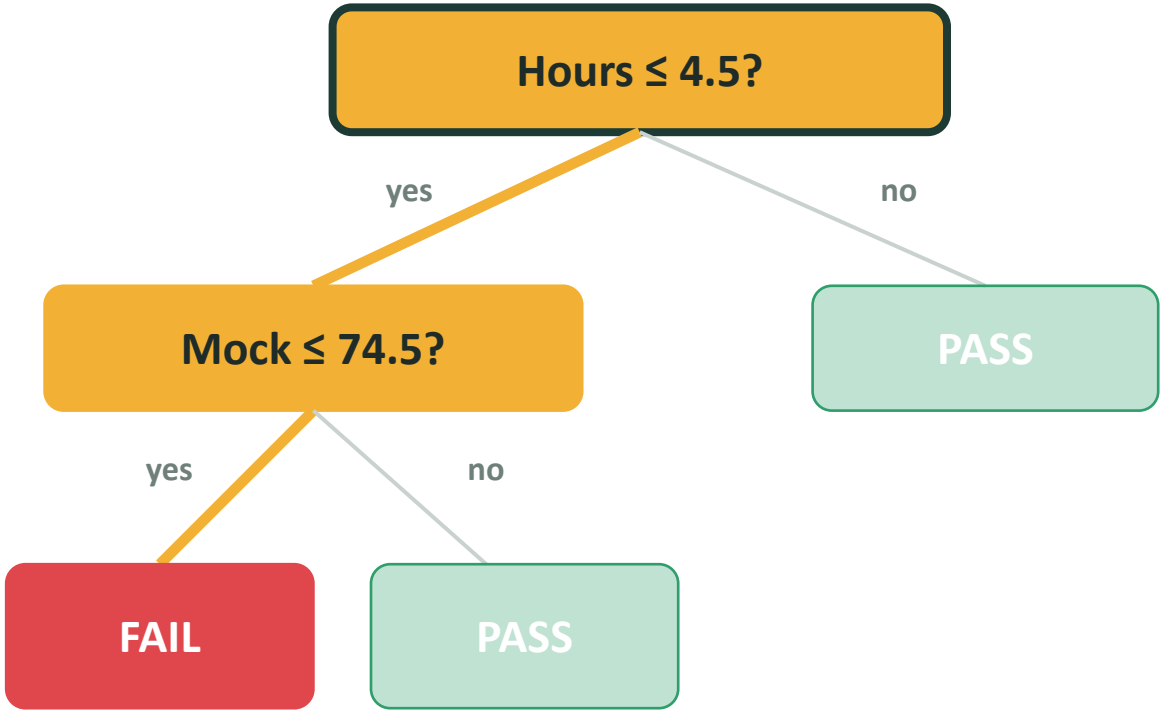
≈ 37% left out

Why $(11/12)^{12}$? Each draw misses a given student with chance 11/12. Twelve independent draws: multiply twelve times.

Keep the left-outs in mind. They are called out-of-bag students, and in Stop 3 they give us a free accuracy check.

Sample B grows a very different tree

Trained on sample B, a fully grown tree gives Riya the opposite answer.



R **4 h yes, mock 70 yes, so FAIL**

Why so gloomy?

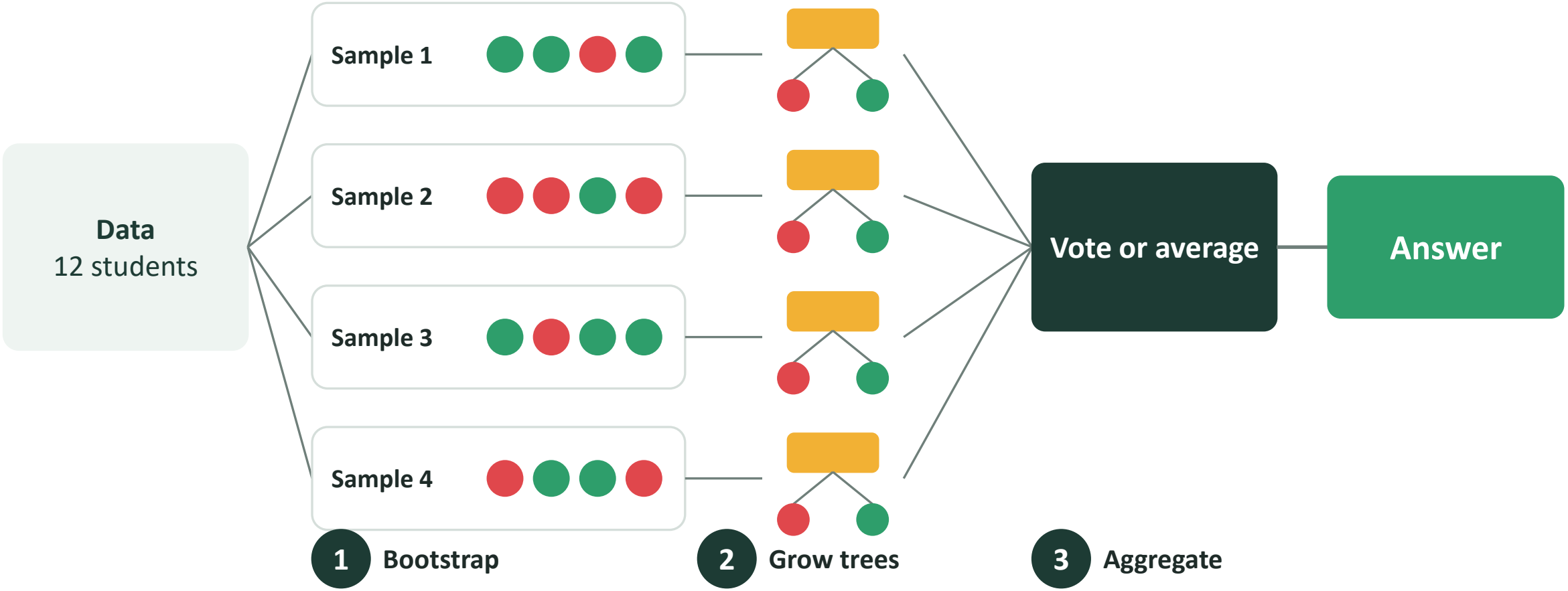
Sample B never saw student 5 (3 hours, mock 68, passed), the student most like Riya.

It saw student 3 (2 hours, mock 72, failed) three times.

One tree, one biased sample, one wrong-looking answer. A forest will out-vote it.

Bagging = Bootstrap AGGregatING

Many samples, many trees, one combined answer.



Your turn: read a bootstrap sample

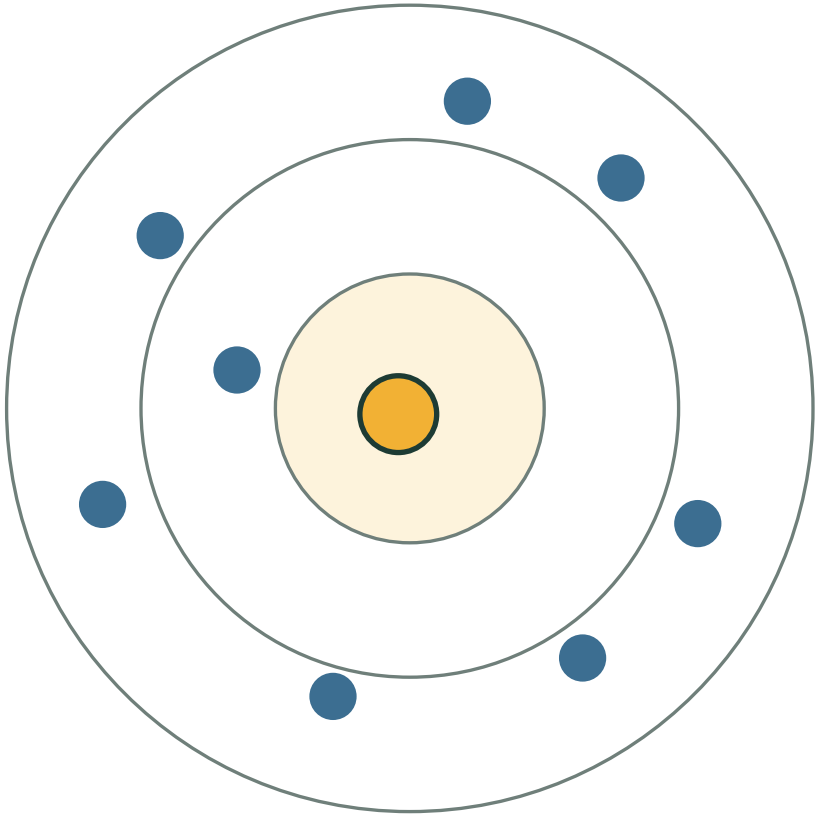
Two minutes. Here is another sample of our 12 students.



1. How many different students are in this sample?
2. Which students are out-of-bag (never drawn)?
3. Which students count three times as much as normal?
4. Would a tree grown on this sample be optimistic or pessimistic? Why?

Why bagging loves deep trees

A deep tree overfits its own sample. Averaging many of them cancels the quirks.



Blue: single trees. Amber: their average.

Each deep tree

Aims at the right place on average (low bias), but scatters a lot (high variance).

The average of many

Keeps the good aim and shrinks the scatter. So forests use fully grown, unpruned trees.

Stop 3 of 5

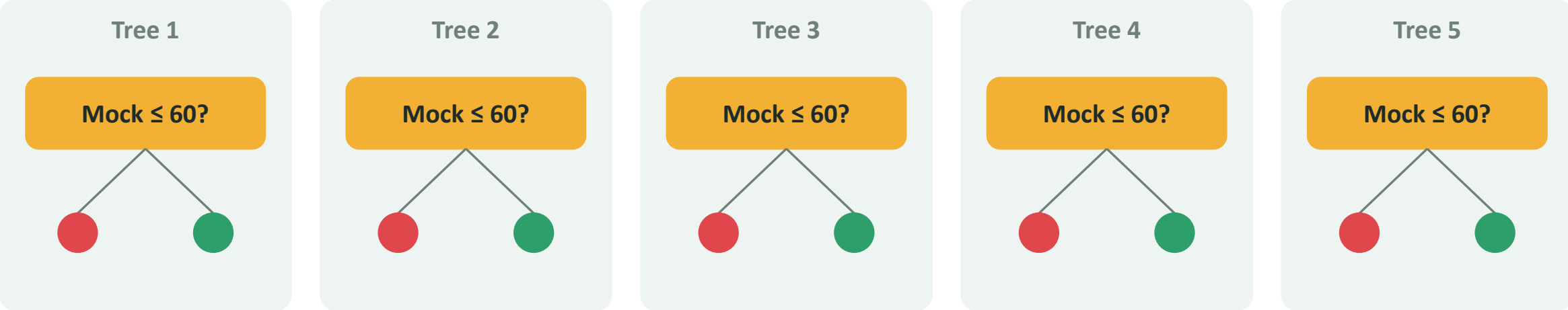
What makes a forest random?

3

Bagging gives each tree different students. A random forest also gives each split different features.

Bagging alone can grow look-alike trees

Imagine a dataset with 20 features, where one feature (mock score) is far stronger than the rest.



Different samples, same first question. The trees end up alike, so they make the same mistakes, and the vote gains little. Remember the five copies?

The twist: random features at every split

At each question, a tree may only consider a random handful of the features.

A dataset with 16 features



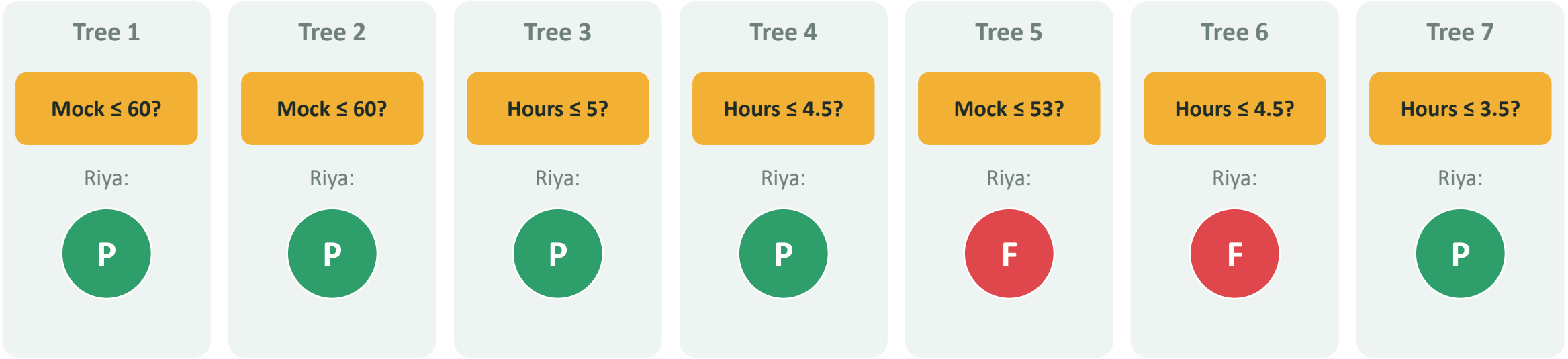
This split may only use the 4 amber features: √16 = 4. The next split draws a fresh 4.

Our 12 students: only 2 features, and √2 rounds down to 1. So every question picks hours or mock score at random, then finds the best cut on that one feature.

The effect: a strong feature can't dominate every tree, so the trees disagree more and the vote improves. Controlled by max_features.

The forest votes on Riya

Seven trees, each with its own sample and random features. Look at their first questions.



5 PASS, 2 FAIL

The forest predicts PASS, with 5/7 = 71% of the votes.

Tree 6 is the sample-B tree. It still says FAIL, but it's out-voted.

The random forest recipe

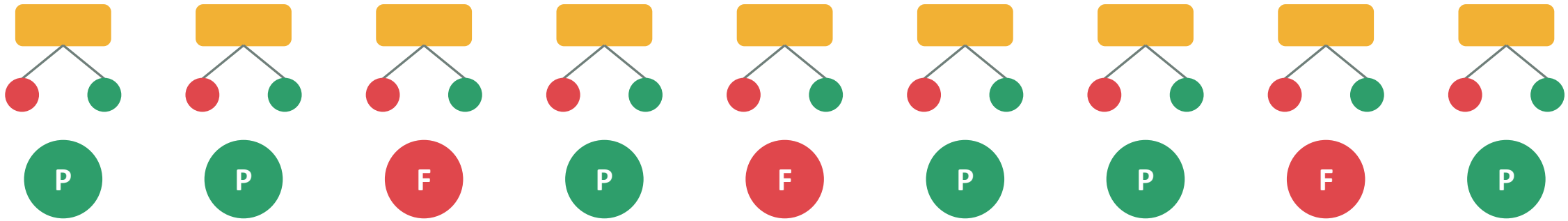
Bagging plus random features at every split.

1	**Bootstrap**	Draw a random sample of the rows, with replacement.
2	**Grow a deep tree**	At every split, consider only a random subset of features (max_features).
3	**Repeat**	Build many trees this way, e.g. 100 or more. Each one is independent.
4	**Combine**	Classification: average the trees' probabilities, pick the top class. Regression: average the numbers.

Random Forest = bagged deep trees + random features at each split

Your turn: count the votes

A forest of 9 trees predicts for a new student.



1. What does the forest predict?
2. What share of the votes does that class get?
3. Tree 3 says FAIL. Should we trust tree 3 or the forest, and why?

Out-of-bag: a free test for every student

Each student is skipped by about a third of the trees. Ask only those trees about them.

5

Student 5
3 hours, mock 68, actually passed

Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Tree 6	Tree 7
trained on student 5	*trained on student 5*	skipped 5 F	*trained on student 5*	skipped 5 F	skipped 5 F	skipped 5 F

Out-of-bag vote: 4 FAIL, 0 PASS. Wrong!

Repeat for every student and you get OOB accuracy: an honest score, no test set used. oob_score=True

Stop 4 of 5

Random forests for numbers

4

Same forest, but each tree predicts a number and the forest takes the average.

New question: what mark will Riya get?

Same 12 students, now with their actual final exam marks. The pass mark is 50.

ID	Hours	Mock	Final mark
S01	1	45	32
S02	2	55	41
S03	2	72	46
S04	3	50	38
S05	3	68	55
S06	4	58	47
S07	4	77	63
S08	5	62	58
S09	6	56	57
S10	6	84	74
S11	7	70	71
S12	8	90	86

R

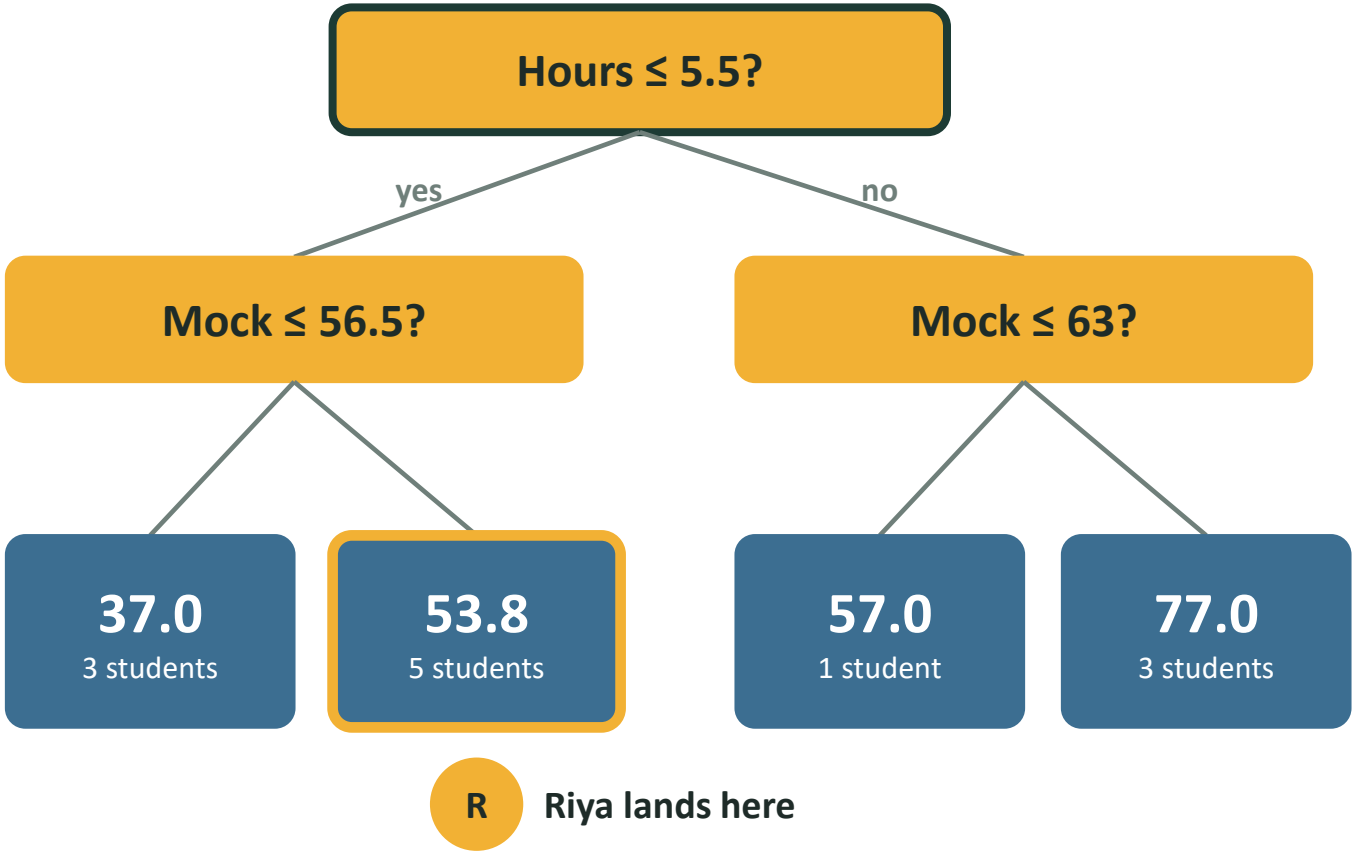
Riya: 4 hours, mock 70

Final mark = ?

Predicting a number is regression. We need trees whose leaves hold numbers, and a forest that combines numbers.

A regression tree: each leaf holds an average

A depth-2 tree grown on the marks.



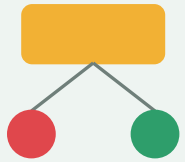
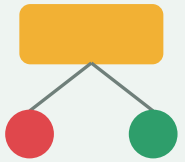
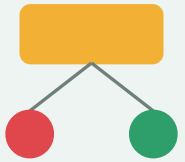
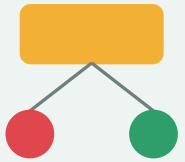
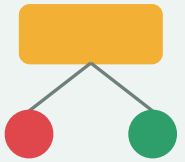
Leaf = average mark

```
53.8 = (46 + 55 + 47 + 63 + 58) ÷ 5
```

Picking questions: instead of Gini, choose the cut that keeps the marks in each group closest to their own average (lowest squared error).

The forest takes the average

Five regression trees, each trained on its own bootstrap sample, predict Riya’s mark.

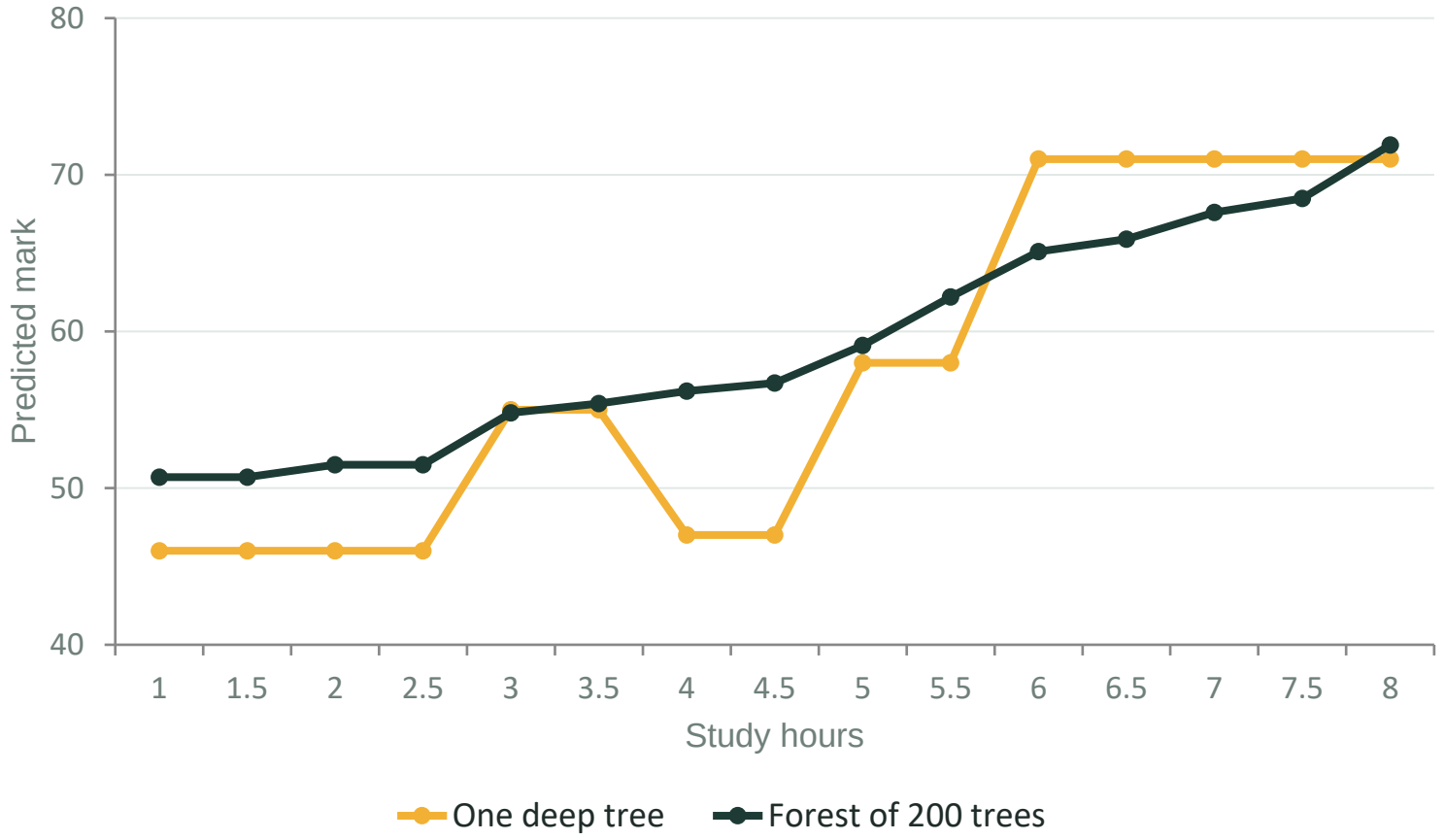
Tree 1	Tree 2	Tree 3	Tree 4	Tree 5
				
46	63	47	74	63

(46 + 63 + 47 + 74 + 63) ÷ 5 = 58.6

Single trees range from 46 to 74. The average is much steadier. With 200 trees it settles at 56.3.

One tree is jumpy. A forest is smooth.

Predicted final mark as study hours change, with the mock score fixed at 70.



At 4 hours, the single tree drops to 47, below its own prediction for 3 hours.

The forest rises steadily, as common sense expects.

Riya: tree 47, forest 56.

Your turn: forest regression

Two minutes.

Question 1

Four trees predict a student's mark:

52	**61**	**58**	**49**

What does the forest predict? Will the student pass?

Question 2

A new student studies 10 hours a day and scores 95 in the mock. The highest mark in our training data is 86.

Could the forest ever predict 95 for them? Why or why not?

Stop 5 of 5

Random forests in scikit-learn

5

Check our examples in code, learn the key settings, then try it on the real dataset.

Riya, one last time

Classifier and regressor on the same 12 students, 200 trees each.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import RandomForestRegressor

X = [[1,45],[2,55],[2,72],[3,50],[3,68],[4,58],
     [4,77],[5,62],[6,56],[6,84],[7,70],[8,90]]
passed = [0,0,0,0,1,0,1,1,1,1,1,1]
marks  = [32,41,46,38,55,47,63,58,57,74,71,86]

clf = RandomForestClassifier(n_estimators=200, random_state=0)
clf.fit(X, passed)
print(clf.predict_proba([[4, 70]]))   # Riya

reg = RandomForestRegressor(n_estimators=200, random_state=0)
reg.fit(X, marks)
print(reg.predict([[4, 70]]))         # Riya
```

Output

```
[[0.15 0.85]]
[56.25]
```

	One tree	Forest
Pass?	Pass, 67%	**Pass, 85%**
Mark	47	**56.25**

Same API as every model so far: fit, predict, predict_proba.

The settings that matter

Defaults in RandomForestClassifier and RandomForestRegressor.

Setting	What it controls	Default	Tip
`n_estimators`	Number of trees	100	More trees: steadier, slower. Doesn't cause overfitting
`max_features`	Features tried at each split	"sqrt" (clf), 1.0 (reg)	Lower: more diverse trees
`max_depth`	Size of each tree	None (grow fully)	Limit if trees memorise noise, or for speed
`min_samples_leaf`	Smallest allowed leaf	1	Raise to smooth predictions
`oob_score`	Out-of-bag accuracy	False	Set True for a free check
`n_jobs`	CPU cores used	None (1 core)	-1 uses all cores; trees train in parallel

Lab: can a forest beat the single tree?

The 240-student dataset from last lesson. The tuned tree scored 68.3% on its test set.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("Decision_Tree_Student_Success_Dataset.csv")
X = df[["hours", "mock_score"]]
y = df["passed"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

forest = RandomForestClassifier(n_estimators=300, oob_score=True,
                                n_jobs=-1, random_state=42)
forest.fit(X_train, y_train)
print("OOB accuracy :", forest.oob_score_)
print("Test accuracy:", forest.score(X_test, y_test))
```

Challenges

Compare OOB and test accuracy. Are they close?

Try min_samples_leaf = 5 and 10. What changes?

Plot test accuracy for n_estimators = 1, 10, 50, 100, 300.

Which features does the forest rely on?

Feature importance: how much each feature reduced impurity, summed over every split in every tree.



Our 200-tree forest on the 12 students. Importances add up to 1.

```
pd.Series(forest.feature_importances_,
          index=X.columns).sort_values()
```

Why it's useful: a forest has hundreds of trees, so you can't read it like a flowchart. Importance is the summary you can show a manager.

Careful: it can favour features with many distinct values, and it says nothing about direction. Double-check with permutation_importance.

One tree or a forest?

Forests usually win on accuracy and stability. Trees win on simplicity.

	One decision tree	Random forest
Accuracy on new data	Often lower	**Usually higher**
Stability	Changes with small data changes	**Steady**
Explaining it	**Read the flowchart**	Hundreds of trees: use feature importance
Speed	**Fast**	Slower, but trains in parallel
Needs feature scaling	No	No
Predicts beyond training range	No	No